



School of Artificial Intelligence
Amrita Vishwa Vidyapeetham, Ettimadai, Coimbatore

22AIE305 — Introduction to Cloud Computing

Course Project Report

DiagramForge

A Cloud-Native DevSecOps Platform for
AI-Assisted Diagram Generation

Architecture, Implementation, and CI/CD Pipeline Report

Team D11

Name	Roll Number
Mahakisore M	CB.SC.U4AIE24333
Yashwanth B	CB.SC.U4AIE24360

Under the guidance of
Dr. Keerthika

September 27, 2026

Abstract

DiagramForge (repository name `Diagram-Ops`) is a cloud-native learning platform built around a deliberately small application: a user describes a system in natural language, a large language model (LLM) converts that description into Mermaid diagram syntax, and the browser renders it. The application is intentionally minimal so that the engineering effort can concentrate on the platform that surrounds it — containerisation, infrastructure as code, Kubernetes orchestration, a security-gated CI/CD pipeline, and (in later phases) observability, progressive delivery, and disaster recovery. This report documents the system as an industry-style architecture and implementation record: functional and non-functional requirements; the application’s internal design (context, use case, component, data, control-flow, and state views); the infrastructure topology on AWS; and, in particular detail, the Jenkins-based DevSecOps pipeline that gates every change behind static analysis (SonarQube), software-composition analysis (OWASP Dependency-Check), and container image scanning (Trivy) before an image may reach the container registry. Real defects found while hardening that pipeline are documented as case studies, including a false-positive CVE caused by an NVD CPE vendor/product collision between the MongoDB server and its Node.js driver, and a class of image vulnerabilities caused by an unused `npm` CLI left in the runtime container.

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Objectives	2
1.3	Scope and Guiding Principle	2
2	Requirements Engineering	2
2.1	Actors	2
2.2	Functional Requirements	3
2.3	Non-Functional Requirements	4
3	System Architecture	4
3.1	System Context	4
3.2	High-Level Runtime Architecture	5
3.3	Layered Backend Component Architecture	5
3.4	Data Model	6
4	Application Behaviour	7
4.1	Use Case Diagram	7
4.2	Control Flow: Generation, Failover, and Cost Guard	7
4.3	Diagram Lifecycle	7
5	DevSecOps Pipeline and Infrastructure	9
5.1	Infrastructure as Code	9
5.2	CI/CD Pipeline Architecture	9
5.3	Security Gates in Detail	10
5.3.1	Case Study: A False-Positive CVE from a CPE Vendor Collision	11
5.3.2	Case Study: A Real Dependency Vulnerability	11
5.3.3	Case Study: An Unused <code>npm</code> CLI as Image Attack Surface	11
5.4	Deployment Architecture	11

6	Implementation Status	12
6.1	Phase Roadmap	12
6.2	Verified Outcomes	13
7	Security Considerations	13
8	Cost Model	14
9	Related Work	14
10	Future Work	15
11	Conclusion	15
	References	15

List of Figures

1	System context diagram	4
2	High-level runtime architecture on Amazon EKS	5
3	Layered backend component architecture	6
4	Entity–relationship model (crow’s-foot notation, logical model enforced by Mon- goose)	6
5	Use case diagram	7
6	Generation, failover, and cost-guard control flow	8
7	Diagram lifecycle state diagram	9
8	CI/CD pipeline with DevSecOps gates (backend service)	10
9	Deployment architecture on AWS	12

1 Introduction

1.1 Motivation

Most student DevOps projects struggle for one of two opposite reasons: the application is either too trivial to justify a real platform, or too complex to finish. DiagramForge resolves this tension deliberately by keeping the *application* small — three persisted entities (users, diagrams, provider usage) and a single LLM-backed generation endpoint — while growing the *platform* around it, phase by phase, until its operational practices resemble what a small production team would actually run: infrastructure as code, a security-gated pipeline, GitOps delivery, observability, and explicit cost discipline.

1.2 Objectives

- Build a working three-tier application (React, Express, MongoDB) with real authentication, real LLM integration, and automated tests.
- Provision every piece of cloud infrastructure through Terraform, never through manual console operations.
- Enforce a non-negotiable quality bar before any container image reaches the registry: static analysis, dependency vulnerability scanning, and image vulnerability scanning, each able to fail the build independently.
- Run the application on Kubernetes (Amazon EKS), delivered through GitOps rather than direct `kubectl` access by a human.
- Treat cost as a first-class constraint — every phase’s monthly cost is known and budgeted in advance.

1.3 Scope and Guiding Principle

The project’s own guiding principle is deliberately blunt: “*Keep the app simple. Make the platform serious.*” This report follows the same split. Section 4 covers the application’s internal design; Section 5 covers the platform, which is the actual subject of the project.

2 Requirements Engineering

2.1 Actors

Actor	Description
Guest	An unauthenticated visitor who may register or log in.
Registered User	An authenticated user who generates, saves, edits, and exports diagrams.
LLM Provider (system)	Groq (primary) or Anthropic Claude (fallback); external systems the backend calls to convert text to Mermaid syntax.
Scheduler (system)	Triggers scheduled operational jobs such as nightly database backups.
Operator / SRE	Interacts with the platform only through Git commits and monitoring dashboards — never directly with the running application.

Table 1: System actors

2.2 Functional Requirements

The full specification enumerates requirements as **FR-*<area><n>***, prioritised with RFC 2119 keywords (**MUST**, **SHOULD**, **MAY**). Table 2 summarises each area and calls out the individual requirements that most directly shaped the design in Section 4.

Area	Count	Architecturally significant requirements
Authentication (FR-A*)	9	Registration and login with bcrypt password hashing (cost ≥ 12); JWT-protected routes return 401 when the token is missing, invalid, or expired.
Generation (FR-G*)	9	FR-G4: on a syntax/type mismatch, retry <i>once</i> , using the <i>same</i> provider that responded (a “sticky retry”) — a formatting glitch never escalates to the paid provider. FR-G5: automatic failover to the secondary provider. FR-G6: a hard daily call cap on the paid provider, enforced <i>before</i> the call. FR-G7: generation never persists anything; the user must explicitly save.
Diagram management (FR-D*)	8	FR-D7: a user may only read, update, or delete their own diagrams; any attempt on another user’s diagram returns 404 rather than 403, so the response never confirms that the resource exists.
Export (FR-E*)	3	SVG download, PNG download, copy raw Mermaid source.
Operational (FR-O*)	6	FR-01/FR-02: separate liveness (<code>/healthz</code>) and readiness (<code>/readyz</code>) probes, so a transient database blip cannot make Kubernetes restart every pod at once. FR-06: graceful shutdown on <code>SIGTERM</code> .

Table 2: Functional requirement areas

2.3 Non-Functional Requirements

Category	Key requirement	Verification
Performance (NFR-P*)	95% of generations complete within 10 s; 95% of non-LLM requests within 300 ms.	Prometheus latency histograms (Phase 9).
Availability (NFR-A*)	99% non-5xx SLO over a rolling 7 days; RTO ≤ 45 min, RPO ≤ 24 h.	Recording rules, burn-rate alerts, timed restore drills.
Security (NFR-S*)	No secret in Git, image layers, or logs; non-root containers. A build fails on any dependency with CVSS ≥ 8, or any fixable HIGH/CRITICAL image vulnerability (NFR-S5).	OWASP Dependency-Check and Trivy gates (Phase 6 — this report’s primary subject).
Scalability (NFR-SC*)	Backend scales from 3 to 10 pods above 70% average CPU; stateless backend.	HPA plus load test.
Observability (NFR-O*)	Per-route rate, errors, and latency visible; logs survive restarts and are queryable by request ID.	Grafana, Loki (Phases 9, 15).
Cost (NFR-C*)	Total AWS spend under \$40/month with disciplined teardown.	AWS Budget alert at 60%.

Table 3: Non-functional requirement categories

3 System Architecture

3.1 System Context

Figure 1 places the system in its operating environment. The Operator never touches the running application directly: they interact only through Git, which drives the pipeline, and through dashboards, which are read-only. This separation is the architectural point of GitOps — the cluster’s desired state lives in a repository, not in anyone’s memory of commands they once ran against it. The dashed edge to the Anthropic API is the failover path; it is exercised rarely by design, which is exactly why it has a dedicated automated test.

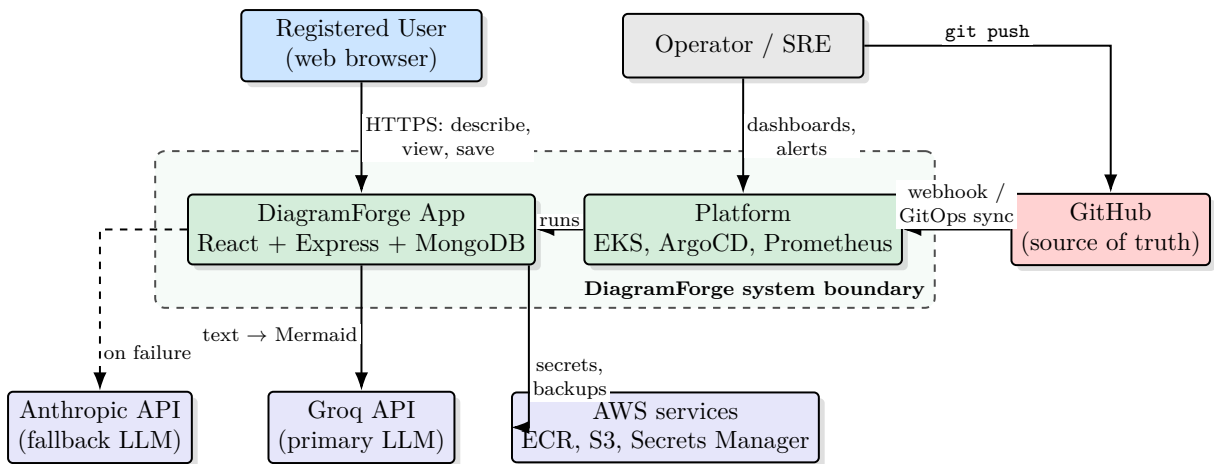


Figure 1: System context diagram

3.2 High-Level Runtime Architecture

Figure 2 shows the target runtime topology on Amazon EKS. The cluster itself is provisioned and verified (Phase 4); the application workloads are deployed onto it through Helm and ArgoCD in Phases 7–8, and today the same three-tier stack runs locally under Docker Compose. The frontend pod runs nginx, which serves the built React bundle and reverse-proxies `/api/*` to the backend service. The frontend calls `/api/...` as a *relative* path, so the browser only ever talks to a single origin: no CORS configuration is needed, and no API URL is baked into the bundle. This matters because Vite resolves environment variables at *build* time, not run time — a baked-in absolute URL would tie one image to one environment, which is the most common way this class of architecture breaks.

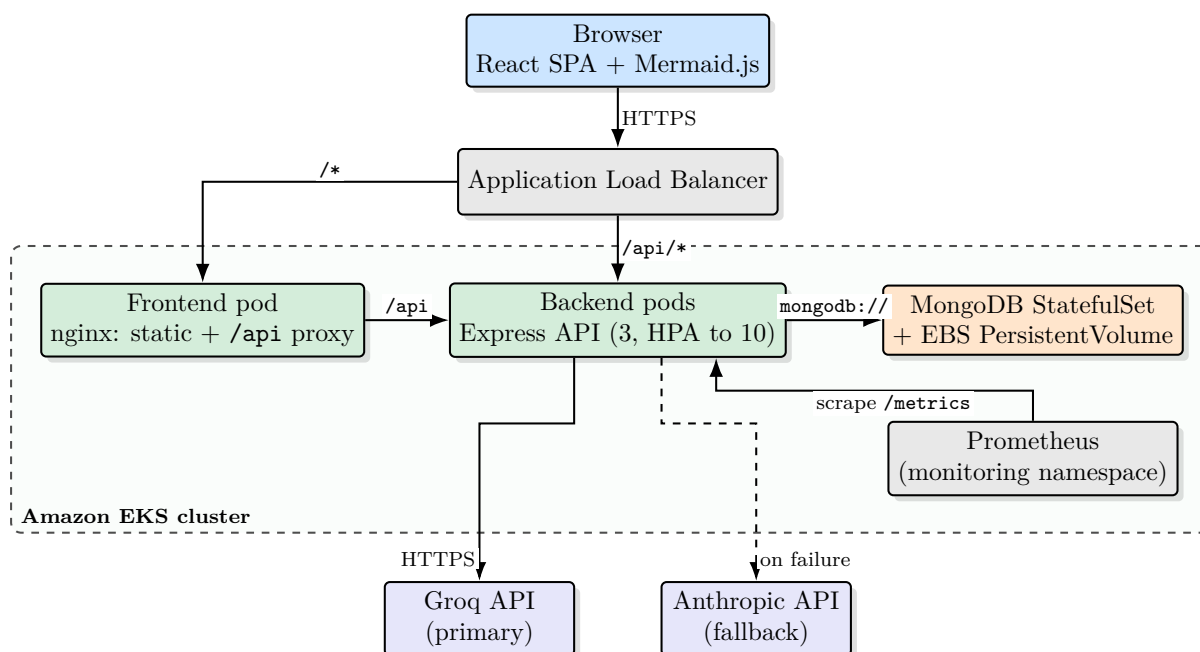


Figure 2: High-level runtime architecture on Amazon EKS

3.3 Layered Backend Component Architecture

The backend follows a strict layering discipline (Figure 3): controllers are thin and never touch models directly, all business logic lives in services, and the choice of LLM provider is hidden behind an interface. `ILLMProvider` is the architecturally important abstraction — `ProviderRegistry` depends on the *interface*, never on `GroqProvider` or `ClaudeProvider` directly (Dependency Inversion). That buys three concrete properties: adding a third provider is one new class and no other change; the whole test suite runs against a mocked interface with zero API calls and zero cost; and swapping which provider is primary is a single environment variable.

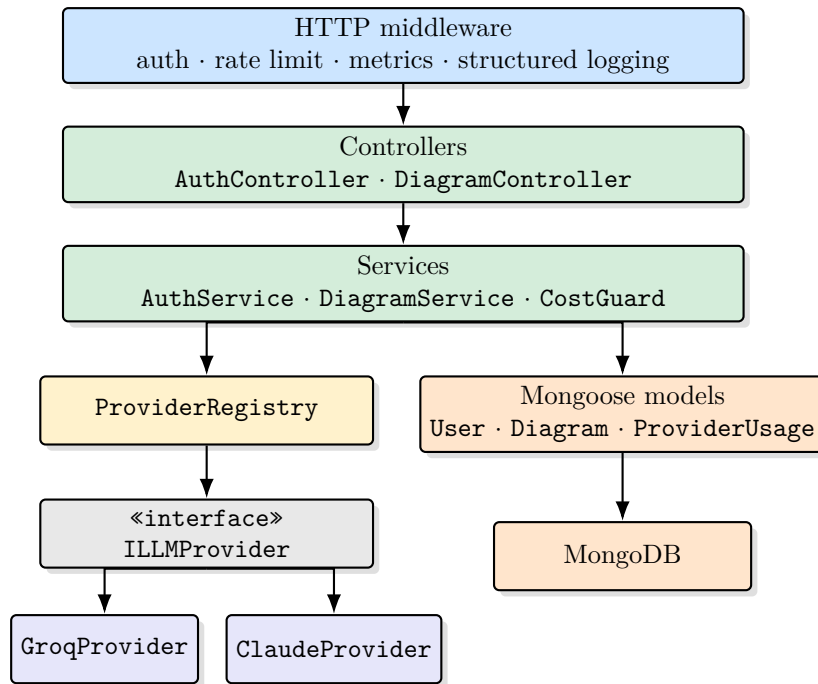


Figure 3: Layered backend component architecture

3.4 Data Model

MongoDB is a document database with no engine-enforced foreign keys, so the entity–relationship view in Figure 4 describes the *logical* model enforced at the Mongoose layer. Referential integrity — for example, removing a user’s diagrams when the user is deleted — is therefore the application’s responsibility, not the database’s. `PROVIDER_USAGE` has no relationship line on purpose: it is a global daily aggregate, not a domain entity, and its unique compound index on `(date, provider)` is what makes the daily cap safe under concurrent requests.

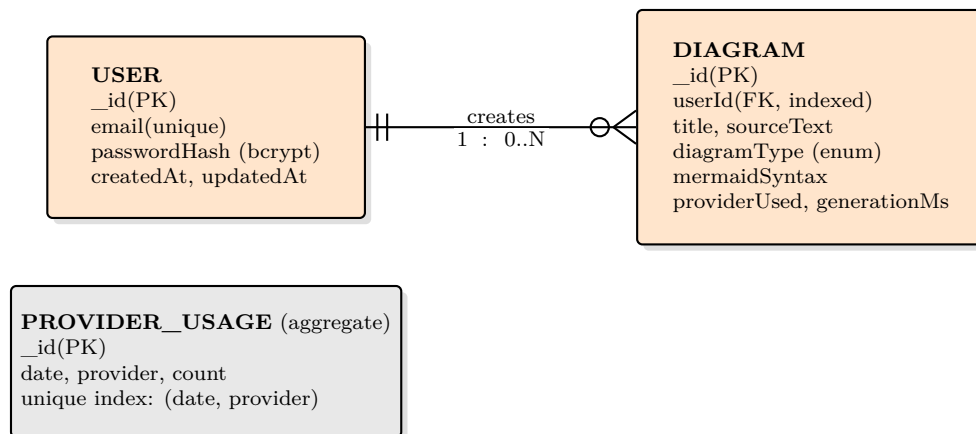


Figure 4: Entity–relationship model (crow’s-foot notation, logical model enforced by Mongoose)

The compound index `{userId: 1, createdAt: -1}` on `DIAGRAM` exists because MongoDB can use a compound index only on a *prefix* of its fields. Every list query filters by `userId` and then sorts by `createdAt`, so `userId` must come first; reversed, the index would be useless for the most common query in the system.

4 Application Behaviour

4.1 Use Case Diagram

In Figure 5, `«include»` means the base use case *always* performs the included one — generating, viewing, and manually editing all necessarily render. `«extend»` means the extension happens *conditionally*: failover extends generation only when the primary provider actually fails.

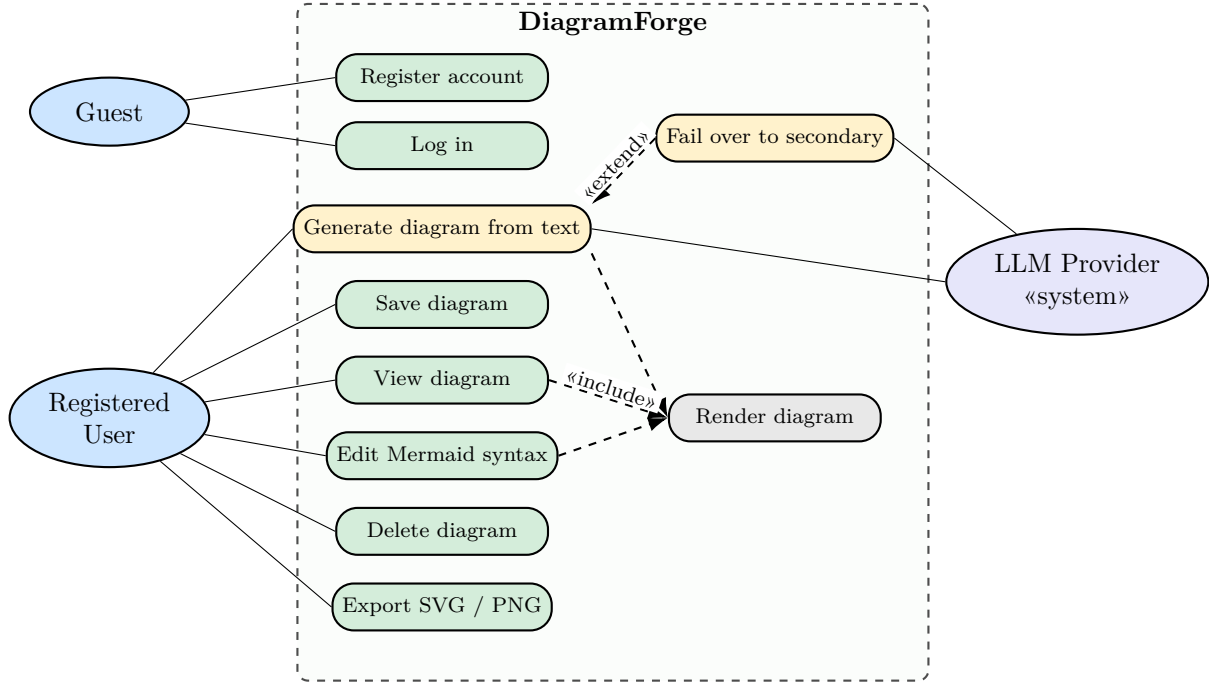


Figure 5: Use case diagram

4.2 Control Flow: Generation, Failover, and Cost Guard

Figure 6 is the most operationally important flow in the system, because it encodes the cost-control logic behind NFR-C2. Three invariants hold by construction, and each was checked against the implementation:

1. **Validation happens before any paid call**, so an oversized input costs nothing.
2. **The retry can happen at most once**. The “already retried?” gate routes to 502 on the second pass, so there is no loop and no unbounded spend.
3. **The daily cap is checked *before* the fallback call, never after**. A cap enforced after the money is spent is not a cap — it is a report.

4.3 Diagram Lifecycle

Figure 7 shows the lifecycle of a diagram from the user’s point of view. The transition worth highlighting is **Rendered → Composing**: discarding a result is a first-class, free action, because generation never persists (FR-G7) — a bad draft leaves no orphaned row to clean up. **SyntaxError** is likewise contained: a Mermaid parse failure renders an inline error, and the page remains usable (FR-D8).

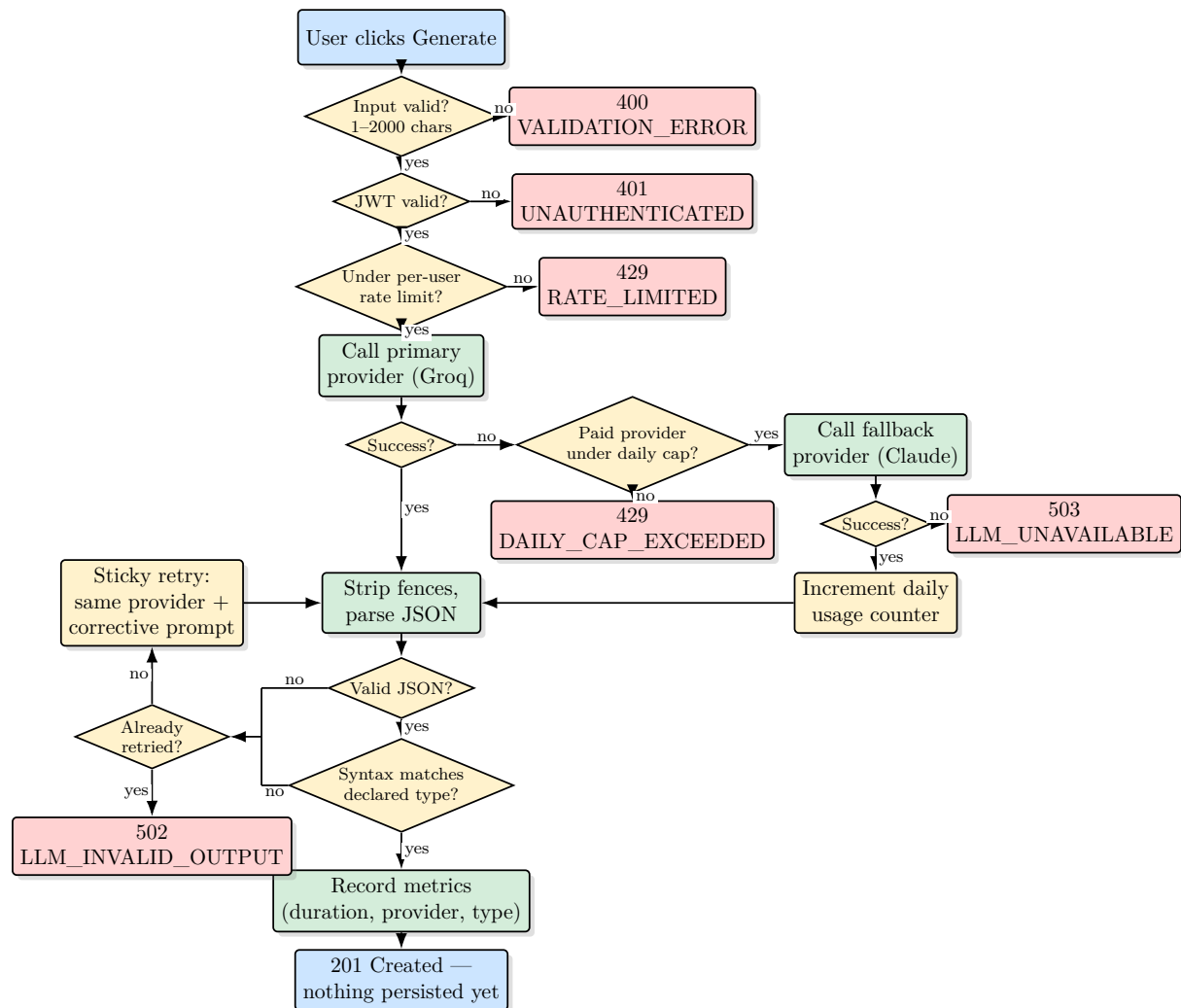


Figure 6: Generation, failover, and cost-guard control flow

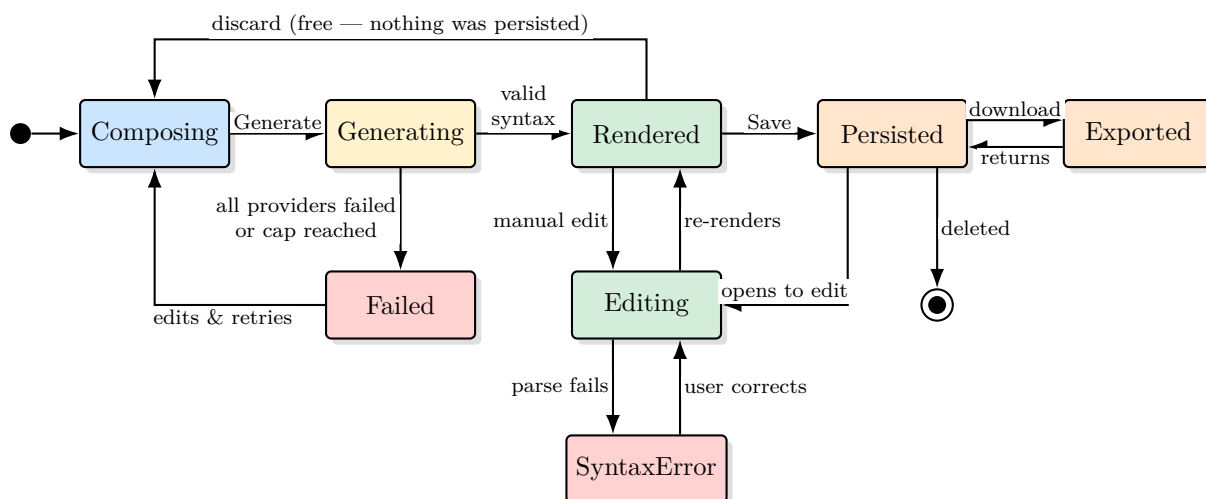


Figure 7: Diagram lifecycle state diagram

5 DevSecOps Pipeline and Infrastructure

This section is the operational centre of the project: every piece of infrastructure is provisioned as code, and every code change passes an automated, security-gated pipeline before it can reach a running environment.

5.1 Infrastructure as Code

All AWS infrastructure — VPC, security groups, IAM roles, the Jenkins EC2 instance, the EKS cluster and its node groups, and the ECR repositories — is defined in Terraform modules under `infrastructure/`. State is stored remotely in S3 and locked through DynamoDB, so it is safe to share and cannot be corrupted by two concurrent applies. Two deliberate cost/security trade-offs are recorded here rather than hidden:

- **Worker nodes sit in public subnets**, avoiding a NAT Gateway (about \$32/month plus data charges). This is a real security trade-off, made for cost and called out explicitly.
- **MongoDB runs as a single-replica StatefulSet**, not a replica set — an honest limitation, recorded for the production-readiness review in Phase 16.

5.2 CI/CD Pipeline Architecture

The pipeline runs on Jenkins as a Multibranch Pipeline job per service. Each DevSecOps gate is implemented once, as a reusable step in a **Jenkins Shared Library** — `sonarQubeScan()`, `owaspDependencyCheck()`, `trivyScan()`, and `pushToECR()` — kept in its own Git repository, because Jenkins can only load a shared library from the root of a repository. Figure 8 shows the backend pipeline; the frontend pipeline is identical apart from the unit-test stage. Any gate can fail the build independently, a failure skips every later stage, and workspace cleanup (`cleanWs()`) always runs.

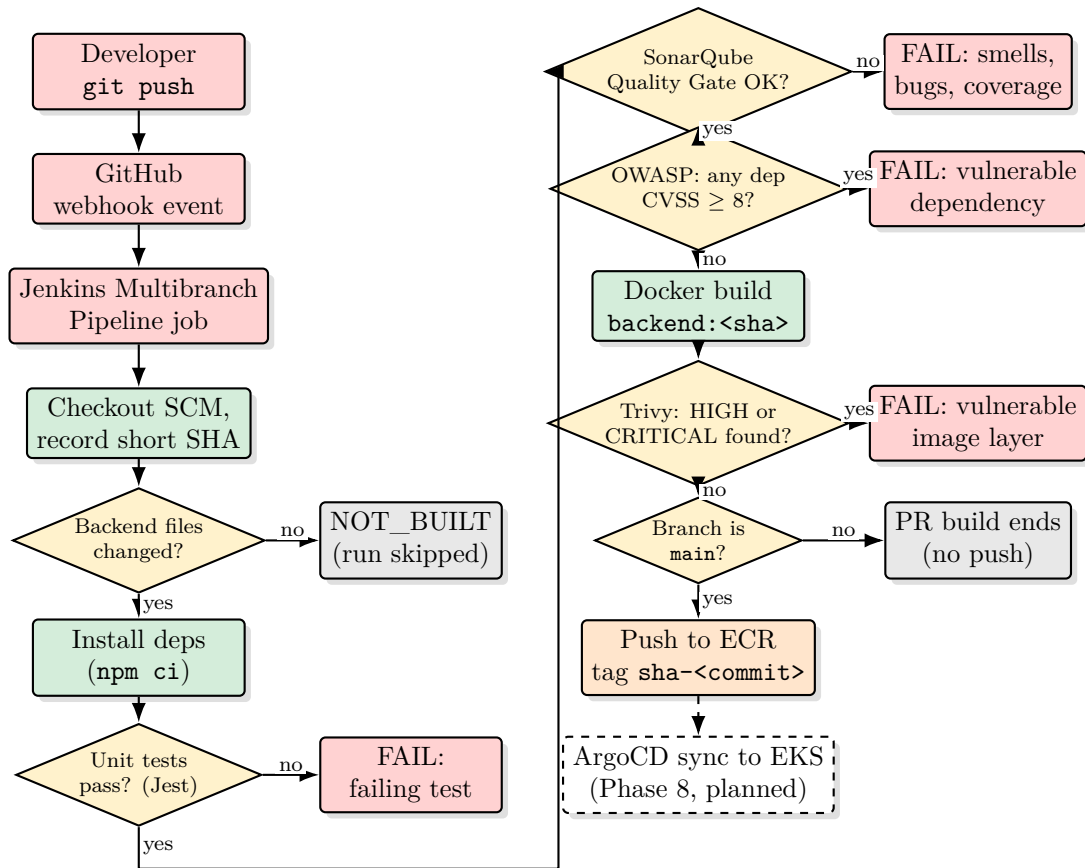


Figure 8: CI/CD pipeline with DevSecOps gates (backend service)

5.3 Security Gates in Detail

Table 4 summarises each gate, the tool behind it, and the requirement it enforces.

Gate	Tool and configuration	What it enforces
Static analysis	SonarQube, via the <code>withSonarQubeEnv</code> and <code>waitForQualityGate</code> pipeline steps	Bugs, code smells, security hotspots, and coverage thresholds on <i>new</i> code. The quality-gate result is delivered back to Jenkins by a SonarQube webhook.
Software composition analysis	OWASP Dependency-Check, <code>--failOnCVSS 8</code> , with a persistent local NVD cache	NFR-S5: the build fails if any dependency carries a known CVE with $CVSS \geq 8$.
Container image scanning	Trivy, <code>--severity HIGH,CRITICAL</code> <code>--exit-code 1</code>	NFR-S5 (image half): the build fails if the built image contains a <code>HIGH</code> or <code>CRITICAL</code> vulnerability, at either the OS-package layer or the language-ecosystem layer.
Registry push	Amazon ECR, immutable tags, tag <code>sha-<commit></code>	Runs only on <code>main</code> , and only after every earlier gate has passed. Tagging by Git SHA keeps every running image traceable to the exact source that produced it.

Table 4: DevSecOps gates and what each one enforces

5.3.1 Case Study: A False-Positive CVE from a CPE Vendor Collision

While hardening the OWASP gate, a finding — CVE-2026-9753, CVSS v3 base score 8.1 — was reported against the `mongodb` npm driver (version 6.20.0, pulled in transitively by `Mon-goose`). Reading the vulnerability’s own affected-software CPE ranges in the raw JSON report, rather than the one-line summary, showed that the CVE actually describes a *MongoDB Server* aggregation-pipeline crash (`$_internalApplyOplogUpdate`), fixed in `mongod` versions 7.0.35, 8.0.24, 8.2.10, and 8.3.3 — different software with an unrelated versioning scheme. NVD’s CPE dictionary uses the same vendor/product identifier (`cpe:2.3:a:mongodb:mongodb`) for both the server and the driver, so CPE-based matching flags any project that merely *talks to* a MongoDB server. Upgrading the driver could never clear the finding; this was confirmed directly, since version 6.21.0 was still flagged. The remediation applied is a suppression rule scoped narrowly to `pkg:npm/mongodb@*` and this single CVE ID, with its justification recorded in the suppression file itself — not a blanket suppression, and not a disabled gate.

5.3.2 Case Study: A Real Dependency Vulnerability

The same investigation surfaced a genuine finding: GHSA-2883-xcg3-v3hh in `js-yaml` 4.3.1 (pulled in by ESLint), a CPU-exhaustion denial of service triggered by YAML merge keys, fixed in 4.3.2. This one was fixed properly, with an npm `overrides` entry scoped to the ESLint dependency chain only, leaving an unrelated `js-yaml` 3.x used by Jest untouched. The two cases illustrate the distinction the gate is designed to force: a finding is either *fixed* or *proven not to apply*, and the proof is written down.

5.3.3 Case Study: An Unused npm CLI as Image Attack Surface

Trivy separately reported four HIGH-severity vulnerabilities in the built backend image, all under `usr/local/lib/node_modules/npm/` — that is, inside the `npm` CLI bundled with the `node:24-alpine` base image, not in the application’s own dependencies, which showed zero findings. The container’s entrypoint invokes `node` directly and never calls `npm` or `npx` at runtime, so `npm` was only ever needed for the image’s `npm ci` build step. The fix removes `npm` from the final image layer once installation completes, which eliminates the unused attack surface rather than merely suppressing the scanner — a stronger remediation, available precisely because the tool was genuinely unnecessary at runtime. It was verified by rebuilding the image, confirming the application still starts and serves `/healthz` with HTTP 200, and re-running Trivy to confirm zero findings.

5.4 Deployment Architecture

Figure 9 shows where each component physically runs once deployment is complete. The Jenkins server, EKS cluster, and ECR repositories exist today; the in-cluster workloads and the ALB routing arrive with Phases 7–8. A deliberate operational habit accompanies this architecture: the EKS cluster and its load balancer are torn down with `terraform destroy` at the end of each working session and rebuilt in roughly fifteen minutes at the start of the next, because the EKS control plane bills \$0.10/hour whether or not any workload is running. Pods pull their images from ECR; the frontend’s pull edge is omitted from the figure for readability.

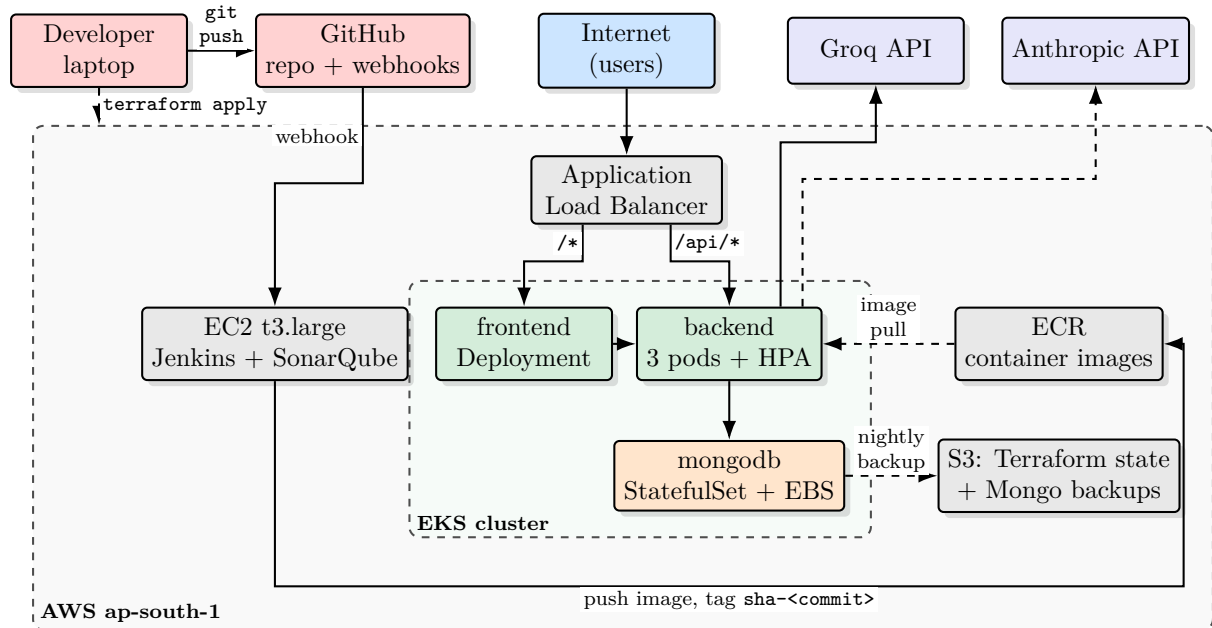


Figure 9: Deployment architecture on AWS

Aspect	Choice	Reasoning
Region	ap-south-1 (Mumbai)	Lowest latency from India; one region everywhere for consistency.
Worker nodes	Public subnets	Avoids a NAT Gateway (about \$32/month); a real, documented security trade-off.
MongoDB	Single-replica StatefulSet	Honest limitation; production would use a replica set or a managed document store.
Jenkins	Single EC2 instance, no HA	It is CI infrastructure, not the product: its downtime blocks deploys, not users.
Backend replicas	3, HPA up to 10	Survives losing one pod with headroom to spare (NFR-A2).
Secrets	Never in Git or images	Fetches at runtime from AWS Secrets Manager via the External Secrets Operator (planned, Phase 12).

Table 5: Deployment design decisions

6 Implementation Status

6.1 Phase Roadmap

The project is executed as incremental phases, each with one concrete learning outcome (Table 6). This report reflects the system during Phase 6, with Phases 0–5 complete and verified.

Phase	Focus	Main learning outcome
0–1	Foundations, application	React, Express, MongoDB, Mermaid, JWT auth, multi-provider LLM with fallback, Docker Compose
2	Jenkins infrastructure	Terraform modules, EC2, VPC, IAM, remote state
3	Jenkins configuration	Configuration as Code, credentials, SonarQube toolchain
4	EKS cluster	Kubernetes on AWS, node groups, IRSA, ALB controller
5	ECR	Private registries, immutable tags, lifecycle policies
6	CI gates	Jenkins pipelines, tests, SonarQube, OWASP Dependency-Check, Trivy (in progress; this report’s primary subject)
7	Helm	Templated manifests, probes, HPA, StatefulSet
8	GitOps	ArgoCD App-of-Apps, auto-sync, self-healing
9	Observability	Prometheus, Grafana, application metrics, alerts
10	DNS and TLS	Route53, ACM certificates, HTTPS through the ALB
11	Persistence and backup	EBS CSI, S3 backups, restore testing
12	Security hardening	External Secrets, NetworkPolicy, RBAC, Pod Security Standards
13	Supply chain and policy	SBOM, image signing, admission policies, IaC scanning
14	Progressive delivery	Argo Rollouts, canary releases, metric-based rollback
15	Logs, traces, incidents	Loki, OpenTelemetry, runbooks, SLOs
16	FinOps and resilience	Budgets, Karpenter, chaos tests, disaster recovery

Table 6: Build roadmap

6.2 Verified Outcomes

The following have been directly verified, not merely designed:

- 46 automated Jest tests across 8 suites (7 unit suites and one integration suite that runs against a real MongoDB instance), plus a real-browser Playwright check of the frontend.
- A passing SonarQube Quality Gate on the backend project, with the result delivered back to Jenkins by webhook.
- A clean OWASP Dependency-Check run (exit code 0) after the remediations described above, completing in under two minutes once analyzers irrelevant to an npm-only project (OSS Index, Maven Central, Yarn audit) were disabled — previously over thirty minutes.
- A clean Trivy scan of the backend image (zero findings) after removing the unused npm CLI.
- Terraform-managed infrastructure with remote state (S3 plus DynamoDB locking), applied and destroyed repeatedly as part of the project’s cost discipline.

7 Security Considerations

- **Secrets never enter version control.** CI secrets (GitHub token, SonarQube token, Anthropic API key, NVD API key) are stored as Jenkins credentials, injected with `withCredentials`, and masked in build logs. A committed `.env.example` documents variable *names* and placeholders only.
- **Least-privilege file permissions.** The server-side secrets file is locked to `600 root:root`; a world-readable copy found during a routine audit was corrected as part of this work.
- **A known residual risk.** Jenkins masks credentials in build *logs*, but a secret passed as a command-line argument remains visible in the OS process list to anyone with shell

access on the build host. The NVD key is currently passed this way; moving it to an environment variable read by the tool is a recorded follow-up.

- **IAM permission additivity.** A resource-based policy, such as an ECR repository policy, can only *add* permission; it cannot subtract what a principal’s identity policy already grants. True exclusivity requires an explicit *Deny*.
- **Ownership checks return 404, not 403.** Confirming that a resource exists but is forbidden would let an attacker enumerate valid IDs.
- **Password handling.** `bcrypt` with a cost factor of 12; `User.toJSON()` strips `passwordHash` during serialisation, so the hash cannot leak even through a careless `res.json(user)`.

8 Cost Model

Table 7 shows monthly AWS cost in `ap-south-1`, running continuously versus only during active working sessions.

Resource	Always on	About 15 h/week
EKS control plane (\$0.10/h, billed even when idle)	\$73	pro-rated
2 × t3.medium worker nodes	\$60 (\$18 spot)	pro-rated
Jenkins EC2 (t3.large)	\$60	pro-rated
Application Load Balancer	\$18 + data	pro-rated
EBS volumes	\$5	\$5 (persists)
Route53 hosted zone	\$0.50	\$0.50 (persists)
ECR storage	negligible	negligible
Total	about \$215/month	about \$25–35/month

Table 7: AWS cost, always on versus disciplined teardown

The affordability strategy is enforced by habit rather than tooling: destroy the EKS cluster and stop the Jenkins instance at the end of every session. That is only survivable because the whole cluster is code — rebuilding it by hand through the AWS console, repeatedly, would be impractical. An AWS Budget alert is set at \$40 with a notification at 60%.

9 Related Work

This project sits at the intersection of two established practices rather than proposing a new technique in either. **GitOps** — declarative desired state in Git, continuously reconciled by an in-cluster controller — is the operating model popularised by tools such as ArgoCD and Flux, and is used here to keep human operators out of the direct `kubectl` path. **DevSecOps** — shifting security scanning left into the CI pipeline instead of treating it as a post-deployment audit — is realised through three independent, build-failing gates rather than one monolithic check, so that every failure is attributable to a specific class of defect. The case studies in Section 5 are offered as concrete, reproducible illustrations of a failure mode that is common but rarely documented in detail: CPE-based vulnerability matching that conflates unrelated products sharing a vendor/product name in the NVD dictionary.

10 Future Work

The remaining phases (Table 6) extend the platform toward a production-representative posture:

- **Helm and GitOps** (Phases 7–8): templated charts delivered through ArgoCD’s App-of-Apps pattern, removing the last manual deployment step.
- **Observability** (Phase 9): Prometheus and Grafana dashboards, with alerts derived from the SLOs in Table 3.
- **TLS, backup, and hardening** (Phases 10–12): end-to-end HTTPS, restore-tested backups, Pod Security Standards, and NetworkPolicy enforcement.
- **Supply-chain security** (Phase 13): image signing with Cosign and admission-time policy with Kyverno, so the cluster refuses any image the pipeline did not sign — closing the loop that the current gates open.
- **Progressive delivery and resilience** (Phases 14–16): canary releases with metric-based rollback, chaos testing, and FinOps controls.

11 Conclusion

DiagramForge shows that an instructive DevOps project does not need a complex application — it needs a complete one, wrapped in a platform that is not simplified for the sake of the exercise. The application layer is intentionally small; the pipeline, infrastructure, and security posture around it are not. The case studies in Section 5 were all found by running the gates against real code and reading the underlying report data rather than trusting a summary line, and each was resolved with a documented, individually justified fix. That discipline, more than any single tool choice, is the real subject of this report.

References

- [1] OWASP Foundation. *OWASP Dependency-Check*. <https://owasp.org/www-project-dependency-check/>
- [2] Aqua Security. *Trivy*. <https://trivy.dev/>
- [3] SonarSource. *SonarQube Documentation*. <https://docs.sonarsource.com/>
- [4] National Institute of Standards and Technology. *National Vulnerability Database*. <https://nvd.nist.gov/>
- [5] HashiCorp. *Terraform Documentation*. <https://developer.hashicorp.com/terraform>
- [6] The Kubernetes Authors. *Kubernetes Documentation*. <https://kubernetes.io/docs/>
- [7] Jenkins Project. *Pipeline and Shared Libraries*. <https://www.jenkins.io/doc/book/pipeline/shared-libraries/>
- [8] OpenGitOps (CNCF). *GitOps Principles v1.0.0*, 2021. <https://opengitops.dev/>
- [9] S. Bradner. *Key words for use in RFCs to Indicate Requirement Levels*. RFC 2119, IETF, 1997.